

BRC Documentation: Introduction to Hazel-HPC

Mery Touceda-Suárez

2026-04-13

Table of contents

03. Login node script with GNU Parallel	1
What is GNU Parallel?	1
Example 1: Downloading data from SRA	2
Example 2: Run a list of commands	3
Creating a Command List	3
Running the Commands in Parallel	4
Sending the processes to the background	5

03. Login node script with GNU Parallel

In some HPCs the compute nodes do not have internet access. This is the case of the NCSU Hazel HPC. This poses a challenge for some bioinformatics tasks, such as:

- Downloading databases and data, which is sometimes embedded in a pipeline
- Installing tools that require internet connectivity

When compute nodes lack internet access, these tasks must be performed on the login node. However, remember that we **CANNOT run computationally intensive tasks on the login node**. Some of these processes, like downloading large databases, can be resource-intensive and could violate cluster policies if executed directly on the login node without proper resource management.

To address this challenge, we'll learn how to use **GNU Parallel** to efficiently manage these tasks while respecting login node limitations.

What is GNU Parallel?

GNU Parallel is a **command-line tool that allows you to execute multiple jobs simultaneously while controlling resource usage**. Think of it as a way to run several commands at once, but with intelligent management of how many processes run at any given time. Instead of running tasks sequentially (one after another) or launching them all at once (which could overwhelm the system), GNU Parallel lets you specify how many jobs should run in parallel. For example, if you need to download 100 datasets, you could tell GNU Parallel to download 4 at a time—as soon as one download finishes, it automatically starts the next one until all are complete.

This controlled parallelization is particularly valuable on login nodes, where we need to be mindful of resource consumption while still completing tasks efficiently. By limiting the number of concurrent processes, GNU Parallel allows us to perform necessary downloads and installations **without monopolizing** the login node's resources or violating cluster policies.

It also provides useful features like progress monitoring, automatic retry of failed jobs, and the ability to resume interrupted work—making it an essential tool for managing data-intensive tasks in restricted HPC environments.

Example 1: Downloading data from SRA

Let's see how to use GNU Parallel to download multiple datasets from the Sequence Read Archive (SRA) efficiently and responsibly on the login node.

Basic Setup

First, create a text file listing the SRA accession numbers you want to download. For example, create a file called `sra_accessions.txt`:

```
SRR28473915
SRR28473916
SRR28473917
SRR28473918
SRR28473919
```

Simple Parallel Download

To download these datasets with GNU Parallel, limiting to 3 concurrent downloads:

```
#!/bin/bash

# create some variables for a more clean script
CPUS=3
PARALLEL=/rs1/shares/brc/admin/tools/parallel-20250922/bin/parallel
SRA_CONTAINER=/rs1/shares/brc/admin/containers/images/quay.io_biocontainers_sra-tools:3.2.1--h4304569_1

# load apptainer module
module load apptainer

# run gnu parallel downloads
cat sra_accessions.txt | $PARALLEL -j $CPUS "apptainer exec $SRA_CONTAINER prefetch {}" > output.log 2>
```

Note: Make sure that you use double quotes `"..."` instead of single quotes `'...'` so that the variable `$SRA_CONTAINER` gets expanded to its actual value.

This command reads each accession number from the file and passes it to `prefetch`. The `-j 3` flag ensures only 3 downloads run simultaneously, preventing system overload. By using `> output.log 2> error.log` we redirect the standard output and standard error to separate files.

More Robust Download with Progress Monitoring

For a more informative and robust approach:

```
#!/bin/bash

# create some variables for a more clean script
CPUS=3
PARALLEL=/rs1/shares/brc/admin/tools/parallel-20250922/bin/parallel
JOBLOG=/your_workdir/path/download_log.txt
SRA_CONTAINER=/rs1/shares/brc/admin/containers/images/quay.io_biocontainers_sra-tools:3.2.1--h4304569_1

# load apptainer module
module load apptainer
```

```
# run gnu parallel downloads with joblog
cat sra_accessions.txt | $PARALLEL -j $CPUS --progress --joblog $JOBLOG "apptainer exec $SRA_CONTAINER
```

Key options explained:

- `-j $CPUS`: Run 3 (\$CPUS) jobs in parallel
- `--progress`: Display a progress bar showing completed/remaining jobs
- `--joblog $JOBLOG`: Create a log file (`download_log.txt`) tracking each job's status and timing
- `> output.log 2> error.log`: Redirect standard output and standard error to separate files

Example 2: Run a list of commands

Sometimes you need to execute multiple independent commands that all require internet access. GNU Parallel can read a list of these commands from a file and execute them in parallel on the login node. This is particularly useful for tasks like downloading reference genomes, databases, or datasets from different sources.

Creating a Command List

First, create a file called `all_commands.txt` with your internet-dependent commands. Here's an example with common bioinformatics download tasks:

```
wget https://ftp.ncbi.nlm.nih.gov/genomes/refseq/bacteria/assembly_summary.txt -O bacteria_summary.txt
curl -O https://ftp.ensembl.org/pub/release-110/fasta/homo_sapiens/dna/Homo_sapiens.GRCh38.dna.chromosomes
SRA_CONTAINER=rs1/shares/brc/admin/containers/images/quay.io_biocontainers_sra-tools:3.2.1--h4304569_1
apptainer exec $SRA_CONTAINER prefetch SRR123456
apptainer exec $SRA_CONTAINER prefetch SRR123457
wget https://ftp.ncbi.nlm.nih.gov/blast/db/16S_ribosomal_RNA.tar.gz
curl -O https://ftp.uniprot.org/pub/databases/uniprot/current_release/knowledgebase/complete/uniprot_sprot
aws s3 cp s3://1000genomes/data/sample1.vcf.gz . --no-sign-request
aws s3 cp s3://1000genomes/data/sample2.vcf.gz . --no-sign-request
```

These commands download various bioinformatics datasets and databases from different online repositories using appropriate tools for each source. The list includes bacterial genome summaries and BLAST databases from NCBI (using `wget`), human genome sequences and protein databases from Ensembl and UniProt (using `curl`), sequencing data from the Sequence Read Archive (using `prefetch`), and genomic variant files from the 1000 Genomes Project on AWS S3 (using `aws s3 cp`), all of which will be executed in parallel to efficiently retrieve multiple large files simultaneously.

wget commands

```
wget https://ftp.ncbi.nlm.nih.gov/genomes/refseq/bacteria/assembly_summary.txt -O bacteria_summary.txt
```

- **wget**: A command-line tool for downloading files from the web
- The URL points to a bacterial genome assembly summary from NCBI
- `-O bacteria_summary.txt`: Saves the downloaded file with a specific name (instead of using the original filename)

curl commands

```
curl -O https://ftp.ensembl.org/pub/release-110/fasta/homo_sapiens/dna/Homo_sapiens.GRCh38.dna.chromosomes
```

- **curl**: Another download tool (similar to `wget` but with different features)

- -O: Saves the file with its original filename from the URL
- Downloads human chromosome 1 sequence data in FASTA format from Ensembl

prefetch commands

```
SRA_CONTAINER=/rs1/shares/brc/admin/containers/images/quay.io_biocontainers_sra-tools:3.2.1--h4304569_1
apptainer exec $SRA_CONTAINER prefetch SRR123456
apptainer exec $SRA_CONTAINER prefetch SRR123457
```

- **prefetch:** Part of the SRA Toolkit for downloading sequencing data from NCBI's Sequence Read Archive
- SRR123456 and SRR123457: Accession numbers for specific sequencing datasets
- Automatically downloads and caches the data in the proper format

aws s3 cpcommands

```
aws s3 cp s3://1000genomes/data/sample1.vcf.gz . --no-sign-request
aws s3 cp s3://1000genomes/data/sample2.vcf.gz . --no-sign-request
```

- **aws s3 cp:** AWS command-line tool for copying files from Amazon S3 storage
- s3://1000genomes/data/sample1.vcf.gz: The source file location in S3
- .: Copies to the current directory
- --no-sign-request: Accesses public S3 buckets without AWS credentials (for openly available data like the 1000 Genomes Project)

Running the Commands in Parallel

Now use this script to execute the commands with controlled parallelization:

```
#!/bin/bash
# create some variables for a more clean script
CPUS=6
PARALLEL=/rs1/shares/brc/admin/tools/parallel-20250922/bin/parallel

# load apptainer module
module load apptainer

# run a list of commands
cat all_commands.txt | $PARALLEL -j $CPUS -a - > output.log 2> error.log
```

Script breakdown:

- CPUS=6: Limits execution to 6 concurrent downloads
- PARALLEL: Variable storing the path to the GNU Parallel executable (adjust for your system)
- -j \$CPUS: Run up to 6 jobs simultaneously
- -a -: Read commands from standard input (stdin), which comes from the cat command

Adding Progress Tracking

For better monitoring, enhance the script with progress reporting:

```
#!/bin/bash
CPUS=6
PARALLEL=/rs1/shares/brc/admin/tools/parallel-20250922/bin/parallel
JOBLOG=/your_workdir/path/commands_log.txt
```

```
# load apptainer module
module load apptainer

# Run commands with progress tracking and logging
cat all_commands.txt | $PARALLEL -j $CPUS --progress --joblog $JOBLOG > output.log 2> error.log
```

This approach allows you to download multiple resources from different sources simultaneously while maintaining control over login node resource usage.

Sending the processes to the background

Since we are running these scripts on the login node, we are going to want to be able to send them to the background so we can continue using the login node terminal. There are several options to do this

1. Execute the process directly on the background

```
./simple_download.sh &
```

2. Start the process, then background it

- Run the script: `./simple_download.sh`
- Press `Ctrl+Z` to suspend it
- Type `bg` to resume it in the background

3. Modifying your script

```
# Save the process ID
echo $! > download.pid
echo "Background process ID: $!"
echo "To kill this process later, run: kill $(cat download.pid)"
```

This modification allows you to directly obtain the process ID and send it to the background.

You can also kill the process manually by: 1. Finding the process id with: `ps -ef` 2. Killing the process using its process id with: `kill -9 [process_id]`